Verification and Validation Plan — RDM

Document ID	VVP-001
Revision	750d3eb
Date	2026-11-11
Status	Controlled in git

Contents

1 Purpose 3
2 Intended use 3
3 User needs 3
4 Verification approach 3
5 Validation approach 4

Id: `VVP-001`

1 Purpose

Defines how RDM is verified (does it meet its requirements, the acceptance criteria?) and validated (does it meet the user needs / intended use?). The vocabulary is the repository's `CONTEXT.md`.

2 Intended use

RDM audits the record of regulated software — the spec it is held to (in Gherkin), checklists, risks, test results and their git history — and lets the team gate releases on it, check documents against standards, render regulatory documents from it, and query it as a read-only graph. Its users are regulatory authors, engineers, reviewers, and agents working under their direction. RDM is not a medical device and does not itself establish compliance; it keeps the evidence that a team presents.

3 User needs

RDM's user needs are in its spec, written the Löyly way: each is a screen of a context's event model (`.pi/spec/<context>/event-model/area.edn`), its key the need's id (`release/release-blocked`) and its description what the actor needs, linked to the Allium surface that models it (who it faces, the rules it provides, what it guarantees) and to the event-storm events behind it. They are the validation anchors. Every user need must be addressed by a verified requirement before a release, and validated by a person: a validation record in `dhf/validation/` names the need's id.

4 Verification approach

RDM's requirements are the accepted scenarios of its spec (`.pi/spec/<context>/spec.feature`, one directory per bounded context, each a slice of `.pi/plan/`), each a concrete Given/When/Then example stamped `@item-NN-SSS` and tagged with the Allium rule or guarantee it covers, which links it to the user need it addresses. Each is an acceptance criterion, verified by the automated tests in RDM's `tests/acceptance/` that claim it with `@allure.tag("item-NN-SSS")`. The test's verification steps are its own, not criteria: a requirement is accepted as a whole. A user need is met when it is validated and every requirement addressing it is verified; a requirement that is a risk control counts once it is verified and its risk's residual is acceptable. `rdm story release-gate` enforces this.

5 Validation approach

User need	Summative (record of truth)	Formative (supporting)
`publishing/compile-dhf`; `release/release-blocked`; `release/trace-verification`	maintainer review that the compiled DHF, gates, and traceability meet the documented intent	dogfooding: RDM compiles its own DHF (this file set)
`specification/validation-visible`	maintainer review that the release gate names every user need without an approved validation record	dogfooding: RDM's own release gate warns on its unvalidated needs
`compliance/gap-analysis`	maintainer review that gap analysis flags real missing standard references against shipped checklists	dogfooding: `rdm gap` over RDM's own released docs
`specification/scaffold-project`	maintainer review that a scaffolded project builds a release and passes the relevant gap checklists	dogfooding: `fresh_release_test` builds an init'd project end-to-end
`specification/adopt-repository`	maintainer review that an adopted repository ends up with the DHF skeleton and the CI gates and that nothing pre-existing was overwritten	trial adoption into a scratch copy of a real repository; `rdm adopt` acceptance test exercises the skip-not-overwrite contract
`release/release-evidence`	maintainer review that a produced evidence bundle and DMR index are complete and agree with the record they were generated from	dogfooding: `rdm story dmr` / `evidence-bundle` run against RDM's own DHF
`graph/explore-graph`	maintainer review that the projected graph agrees with the record it was built from (the spec's needs and requirements, contexts, documents, results, commits) and is explorable in a graph browser	dogfooding: RDM's own DHF projected, queried, and browsed in AWS Graph Explorer
`graph/agent-reads-record`	maintainer review that an agent harness connected to the server answers traceability questions about RDM's own record correctly and has no way to change it	dogfooding: agent sessions on RDM itself using `rdm graph mcp`
`risk/risk-register`	maintainer review that the gate's risk findings match a manual review of the	dogfooding: RDM's own tool-risk register (in CSV-001, `dhf/`

User need	Summative (record of truth)	Formative (supporting)
	same register with the risk-analysis method	`documents/tool_validation.md`) held to the release gate
`publishing/user-manual`	maintainer review that the manual's pages teach what the gates enforce	dogfooding: RDM's own manual (`dhf/documents/user_manual.md`), published per release

Formative evidence never gates release.